

Langage Python

ISUP, Sorbonne Université

Etienne Guével

Ingénieur de Recherche - SCAI

etienne.guevel@sorbonne-universite.fr

Septembre-Novembre 2025

Partie 3

Librairies scientifiques et visualisation de données

Most common libraries in Data Science

Data manipulation libraries

- Numpy: matrices and multi-dimensional arrays
- Pandas: data manipulation and analysis vis

Data visualization libraries

- matplotlib: unavoidable library
- seaborn: high-level interface of matplotlib

Librairies scientifiques : SciPy, NumPy



Fundamental algorithms for scientific computing in Python

GET STARTED

SciPy 1.8.0 released! 2022-02-05

FUNDAMENTAL ALGORITHMS

SciPy provides algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, statistics and many other classes of problems.

BROADLY APPLICABLE

The algorithms and data structures provided by SciPy are broadly applicable across domains.

FOUNDATIONAL

Extends NumPy providing additional tools for array computing and provides specialized data structures, such as sparse matrices and k-dimensional trees.

PERFORMANT

SciPy wraps highly-optimized implementations written in low-level languages like Fortran, C, and C++. Enjoy the flexibility of Python with the speed of compiled code.

EASY TO USE

SciPy's high level syntax makes it accessible and productive for programmers from any background or experience level.

OPEN SOURCE

Distributed under a liberal [BSD license](#), SciPy is developed and maintained [publicly on GitHub](#) by a vibrant, responsive, and diverse [community](#).

Librairie dédiées au calcul scientifique

NumPy est la librairie de référence pour le calcul scientifique.

- Structure de données représentant des tableaux multidimensionnels
`ndarray`
- Un ensemble très complet de routines permettant des opérations optimisées sur des tableaux
- Exemples : opérations mathématiques et logiques, tris, algèbre linéaire, opérations statistiques, simulations aléatoires, etc.

Documentation :

https://numpy.org/doc/stable/user/absolute_beginners.html

Installation et import

Librairie intégrée au projet Anaconda

```
pip install numpy
```

ou bien

```
conda install numpy
```

Convention d'import

```
import numpy as np
```

Création d'un objet ndarray à partir d'un objet itérable

```
arr = np.array(<iterable>)
```

Par exemple :

Création d'un objet ndarray à partir d'un objet itérable

```
>>> arr = np.array([])
>>> arr = np.array([1, 2])
>>> arr = np.array([[1, 2], [3, 4]])
>>> arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

Lien vers la documentation : [numpy.ndarray.html](https://numpy.org/doc/stable/reference/arrays.ndarray.html)

Attribut shape

```
>>> arr = np.array([])
>>> arr.shape
(0,)
>>> arr = np.array([1, 2])
>>> arr.shape
(2,)
>>> arr = np.array([[1, 2], [3, 4]])
>>> arr.shape
(2, 2)
>>> arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
>>> arr.shape
(2, 2, 2)
```

Indexing et slicing, arrays 1D

```
>>> data = np.array([1, 2, 3])
```

```
>>> data[1]
```

```
2
```

```
>>> data[0:2]
```

```
array([1, 2])
```

```
>>> data[1:]
```

```
array([2, 3])
```

```
>>> data[-2:]
```

```
array([2, 3])
```

Indexing et slicing, arrays > 2D

```
>>> arr = np.array([[1, 2], [3, 4], [5, 6]])
```

```
>>> arr.shape  
(3, 2)
```

```
>>> arr[0, 1]  
2
```

```
>>> arr[1:3]  
array([[3, 4],  
       [5, 6]])
```

```
>>> arr[1:3, 0]  
array([3, 5])
```

Types de données intégrés à NumPy

- Entier 64-bits `np.int64`
- Entier 32-bits `np.int32`
- Flottant 32-bits `np.float32`
- Nombre complexe `np.complex`
- Booléen `np.bool`
- Le type de base `np.object`

Librairies Pandas pour le traitement de données

En bref

Pandas est une librairie performante dédiée à la manipulation de données et à l'analyse de données

Liens utiles

- Documentation officielle : pandas.pydata.org
- Pour commencer : pandas.pydata.org/getting_started

Installation et import

Librairie intégrée au projet Anaconda

```
pip install pandas
```

ou bien

```
conda install pandas
```

Convention d'import

```
import pandas as pd
```

Créer un tableau de données

```
import pandas as pd
data = [['Mary', 968], ['Jessica', 155], ['Bob', 578]]
df = pd.DataFrame(data=data, columns=['Names', 'Births'])
```

Structure de données DataFrame

- Indices sur les lignes et sur les colonnes
- Données de types différents

	Names	Births
0	Mary	968
1	Jessica	155
2	Bob	578
3	John	403
4	Mel	199

Lire un fichier CSV

```
# Create a DataFrame from a CSV file
```

```
df = pd.read_csv("path/to/data.csv")
```

	date	precipitation	temp_max	temp_min	wind	weather	month
0	2012-01-01	0.0	12.8	5.0	4.7	drizzle	January
1	2012-01-02	10.9	10.6	2.8	4.5	rain	January
2	2012-01-03	0.8	11.7	7.2	2.3	rain	January
3	2012-01-04	20.3	12.2	5.6	4.7	rain	January
4	2012-01-05	1.3	8.9	2.8	6.1	rain	January

Décrire les données

```
# Print the first 10 rows
print(df.head(10))

# Describe the data: mean, max, min, median, etc.
print(df.describe())
```

	precipitation	temp_max	temp_min	wind
count	1461.000000	1461.000000	1461.000000	1461.000000
mean	3.029432	16.439083	8.234771	3.241136
std	6.680194	7.349758	5.023004	1.437825
min	0.000000	-1.600000	-7.100000	0.400000
25%	0.000000	10.600000	4.400000	2.200000
50%	0.000000	15.600000	8.300000	3.000000
75%	2.800000	22.200000	12.200000	4.000000
max	55.900000	35.600000	18.300000	9.500000

Sélectionner un sous-ensemble

```
# Use the attribute loc
df.loc[[1, 3], ["date", "weather", "temp_max", "temp_min"]]

# Use ':' for selecting all the rows
df.loc[:, ["date", "weather", "temp_max", "temp_min"]]
```

	date	weather	temp_max	temp_min
1	2012-01-02	rain	10.6	2.8
3	2012-01-04	rain	12.2	5.6

Trier

```
# Sort w.r.t the value of "temp_max"  
df.sort_values(by="temp_max", ascending=True)
```

	date	precipitation	temp_max	temp_min	wind	weather	month
767	2014-02-06	0.0	-1.6	-6.0	4.5	sun	February
18	2012-01-19	15.2	-1.1	-2.8	1.6	snow	January
766	2014-02-05	0.0	-0.5	-5.5	6.6	sun	February
706	2013-12-07	0.0	0.0	-7.1	3.1	sun	December
17	2012-01-18	19.8	0.0	-2.8	5.0	snow	January
...	...	...	...	...	...	...	...
1307	2015-07-31	0.0	34.4	17.8	2.6	sun	July
228	2012-08-16	0.0	34.4	18.3	2.8	sun	August
1306	2015-07-30	0.0	34.4	17.2	3.5	sun	July
1295	2015-07-19	0.0	35.0	17.2	3.3	sun	July
953	2014-08-11	0.5	35.6	17.8	2.6	rain	August

Grouper et combiner

```
# Create a "groupby" object, i.e. split
gb = df.groupby("month")

# Compute the mean values for each group, i.e. combine
gb.mean()

# Or using another combine function f
gb.apply(f)
```

	precipitation	temp_max	temp_min	wind
month				
April	3.128333	15.020000	6.362500	3.524167
August	1.320161	26.112097	14.769355	2.750806
December	5.021774	8.194355	3.325000	3.618548
February	3.734513	9.860177	4.054867	3.786726
January	3.758065	8.229032	2.696774	3.138710
July	0.388710	25.998387	14.197581	2.911290

Visualisation de données :

Matplotlib

En bref

Matplotlib est LA librairie incontournable pour visualiser des données

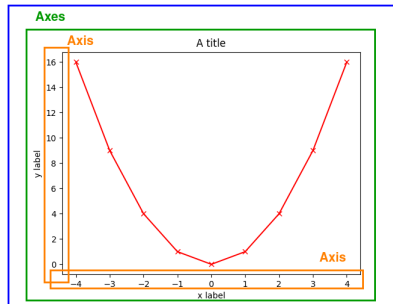
Liens utiles

- Site officiel : matplotlib.org
- *Third party packages* : matplotlib.org/stable/thirdpartypackages
- Exemples : matplotlib.org/stable/gallery
- Tutoriel : github.com/matplotlib/AnatomyOfMatplotlib

Concepts

- Objet de classe “Figure”
- Objet de classe “Axes”
- Plusieurs graphiques sur une même figure : Subplots

Figure



Convention d'import

```
import matplotlib.pyplot as plt
```

Figure

- La “fenêtre” contenant tous les objets composant le ou les graphiques
- Un objet Figure peut contenir : titre, sous-titre, légende, etc.
- Une figure est créée comme suit :

```
fig = plt.figure()
```

Axes

- Dans une figure, on ajoute un ou plusieurs objets **Axes**
- Un tel objet est une **région** de l'image qui recevra les **données à visualiser**
- Une figure peut contenir plusieurs axes
- Un axe contient deux ou trois objets **Axis**

```
import matplotlib.pyplot as plt

fig = plt.figure() # Create the Figure object

# Add an Axe to this figure
ax = fig.add_axes([0.5,0.5,0.8,0.8]) # dimensions [left,
    bottom, width, height] of the new axes

# Plot the distribution of the wind variable in the axe
g1 = ax.hist(x=seattle_weather['wind'])

# Add a title and labels for the x-axis and the y-axis
ax.set_title("Histogram with default parameters")
ax.set_xlabel("Wind")
ax.set_ylabel("Count")

plt.show() # Show the plot
```

```
# Short version (using the fact that figure and axes are
    automatically created)

import matplotlib.pyplot as plt

# Plot the distribution of the wind using a histogram
plt.hist(x=seattle_weather["wind"])

# Add a title
plt.title("Histogram with default parameters")

# Add labels for the x-axis and the y-axis
plt.xlabel("Wind")
plt.ylabel("Count")

plt.show() # Show the plot
```

What is a subplot?

- The `matplotlib.pyplot.subplots` method provides a way to have **multiple plots on a single figure**
- Given the number of rows and columns, it returns a tuple (`fig`, `ax`), giving a single figure '`fig`' with an array of axes '`ax`'

More details:

- educative.io/edpresso/what-is-a-subplots-in-matplotlib
- matplotlib.org/stable/gallery/subplots_axes_and_figures

Usage 1

```
import matplotlib.pyplot as plt

# Create a subplot with 1 row and 2 columns
fig, (ax1, ax2) = plt.subplots(1, 2)

# Hist on the 1st column
ax1.hist(x)

# Scatter on the 2nd column
ax2.scatter(x, y)

# Show the plot
plt.show()
```

Usage 2

```
import matplotlib.pyplot as plt

# Create the figure
fig = plt.figure()

# Create an histogram on the 1st subplot
plt.subplot(121) # or plt.subplot(1, 2, 1)
plt.hist(x)

# Create scatter plot on the 2nd subplot
plt.subplot(122) # or plt.subplot(1, 2, 2)
plt.scatter(x, y)

# Show the plot
plt.show()
```

Title and subtitle

```
import matplotlib.pyplot as plt

plt.hist(x)

plt.title("A title")

plt.subtitle("A title")
```

Legend

```
import matplotlib.pyplot as plt

plt.scatter(x1, y1, label="First scatter plot")

plt.scatter(x2, y2, label="Second scatter plot")

plt.legend(loc=1)
```

Seaborn: high-level interface of matplotlib

Overview of seaborn

Seaborn is a [complementary](#) library for Matplotlib, which provides a high-level [interface](#) to draw [statistical](#) graphics.

It uses Pandas data frames

Useful links

- Gallery: seaborn.pydata.org/examples
- Tutorial: seaborn.pydata.org/tutorial

Seaborn: high-level interface of matplotlib

FIGURE-LEVEL
vs
AXES-LEVEL

RELATIONAL
DISTRIBUTIONAL
CATEGORICAL

Figure-level vs Axes-level functions

Axes-level

Axes-level functions plots data onto a single matplotlib Axes object `ax`

```
# creates a chart in an existing axes object  
sns.histplot(data=df, x="temperature", ax=ax)
```

Figure-level vs Axes-level functions

Axes-level

Axes-level functions plots data onto a single matplotlib Axes object `ax`

```
# creates a chart in an existing axes object
sns.histplot(data=df, x="temperature", ax=ax)
```

Figure-level

Figure-level functions interface with matplotlib through a seaborn object that manages the figure.

For each task, the figure-level function is an interface to various axes-level functions

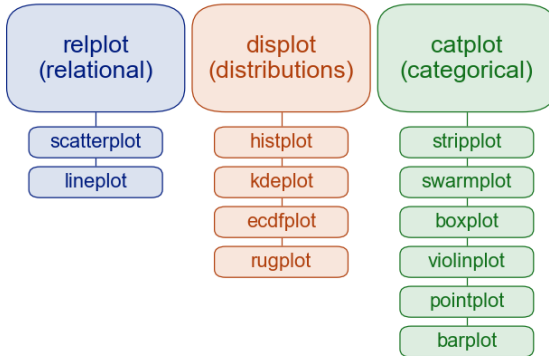
```
sns.displot(data=df, x="temperature", kind="hist")
```

Figure-level vs Axes-level functions

FIGURE-LEVEL

VS

AXES-LEVEL



Same usage / Different tasks

The functions have (almost) the same signature

- data: a pandas DataFrame
- x, y: columns names in the data frame
- hue: name of the column used to add colors
- ax: matplotlib Axes object in which to draw the plot

```
# Scatter plot
```

```
sns.scatterplot(data=df, x="time", y="temperature", ax=ax)
```

```
# Line plot
```

```
sns.lineplot(data=df, x="time", y="temperature", ax=ax)
```

```
# Line plot, Figure-level function
```

```
sns.relplot(data=df, x="time", y="temperature", kind="line")
```

Statistical graphics

Seaborn integrates statistics into the graphics making easier the analysis of the data.

- Automatic computation of statistics: mean, confidence interval, error bars, etc.
- Automatic linear regression fitting

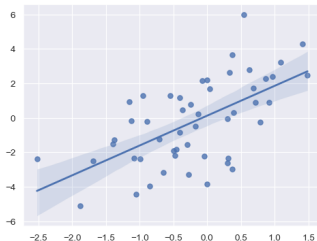
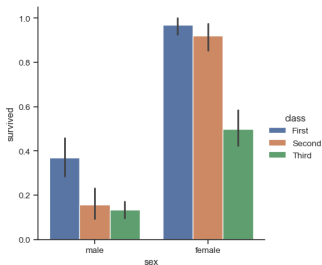


Figure aesthetics

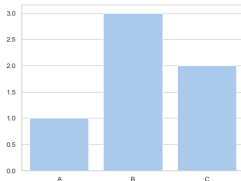
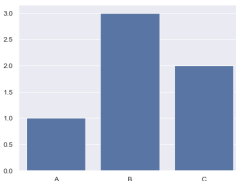
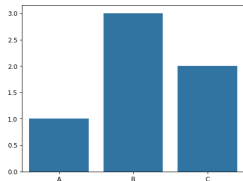
The aesthetics can be modified, helping the communication of insights

The function `set_theme` modifies the background, the color palette, the font, etc.

```
# Default theme
sns.set_theme()

# Dark background with a grid
sns.set_theme(style="darkgrid")

# White background with a grid
sns.set_theme(style="whitegrid", palette="pastel")
```



Some examples

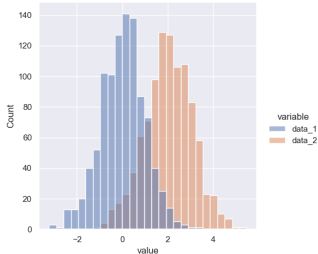
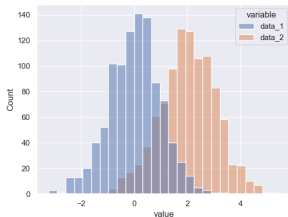
Plotting distributions

```
# Axes-level histogram
```

```
sns.histplot(x="value", hue="variable", data=df)
```

```
# Figure-level histogram
```

```
sns.displot(x="value", hue="variable", data=df)
```



Some examples

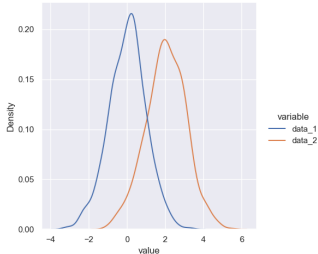
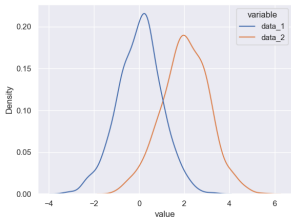
Plotting distributions

```
# Axes-level histogram
```

```
sns.kdeplot(x="value", hue="variable", data=df)
```

```
# Figure-level histogram
```

```
sns.displot(x="value", hue="variable", data=df, kind="kde")
```



Some examples

```
# Pairplot / Matrix of scatterplots
```

```
sns.pairplot(x)
```

```
# Joint plot
```

```
sns.jointplot(x)
```

